EXPERIMENT 8

Shravanya Andhale
D15A-30

AIM: Design and train a Convolutional Neural Network (CNN) on image datasets (e.g., MNIST/CIFAR)

THEORY:

1. Dataset Source

- Dataset: MNIST Handwritten Digit Dataset
- Source: Yann LeCun, Corinna Cortes, and Christopher J. C. Burges
- Accessed through the Keras built-in dataset repository

  Source link: https://www.kaggle.com/datasets/hojjatk/mnist-dataset

The MNIST dataset is one of the most widely used benchmark datasets for image classification and deep learning research.

2. Dataset Description

The MNIST dataset consists of grayscale images of handwritten digits from 0 to 9.

Dataset Characteristics:

- Total images: 70,000
- Training images: 60,000
- Testing images: 10,000

Image properties:

- Image size: 28 × 28 pixels
- Color format: Grayscale
- Pixel values range from 0 to 255

Target Variable:

- Digit labels ranging from 0 to 9 representing handwritten digits.

Before training the CNN model:

- Pixel values were normalized by dividing by 255.
- Images were reshaped into 4D tensors for CNN input format:
  (samples, height, width, channels).

3. Mathematical Formulation of Convolutional Neural Network

A Convolutional Neural Network processes image data using convolution operations to automatically learn spatial features.

Convolution Operation

The convolution layer applies filters to the input image:

Feature Map = Input Image * Kernel

Each filter extracts patterns such as edges, curves, or shapes.

Activation Function

ReLU activation is applied to introduce non-linearity:

$ReLU(x) = max(0, x)$

Pooling Operation

MaxPooling reduces spatial dimensions by selecting the maximum value from a region:

Pooling helps reduce computation and prevents overfitting.

Flatten Layer

Transforms the feature maps into a one-dimensional vector before passing to fully connected layers.

Output Layer

Softmax activation is used for multi-class classification:

$\text{Softmax}(x_i) = \exp(x_i) / \Sigma \exp(x_j)$

This converts outputs into class probabilities.

Loss Function

Sparse Categorical Cross Entropy was used:

$\text{Loss} = -\Sigma\, y_i \log(p_i)$

where p_i is the predicted probability of class i.

4. Algorithm Limitations

- Requires large computational resources for complex architectures
- Training time increases with deeper networks
- Sensitive to hyperparameter choices
- May overfit if regularization techniques are not used

5. Methodology / Workflow

- Import required deep learning and visualization libraries
- Load MNIST dataset using Keras API
- Explore dataset structure and visualize sample images
- Normalize pixel values to the range 0–1
- Reshape image data into 4D tensors for CNN input
- Design CNN architecture with convolution and pooling layers
- Compile model using Adam optimizer and categorical loss function

- Train CNN using training dataset with validation split
- Monitor training and validation accuracy and loss
- Generate predictions on test dataset
- Evaluate model using classification report and confusion matrix
- Visualize model performance and prediction outputs

6. Performance Analysis

CNN Model Results:

Overall Accuracy:
0.99

Classification Report:

- Precision values ranged from 0.98 to 1.00
- Recall values ranged from 0.98 to 1.00
- F1-scores were consistently around 0.99

Class-wise Interpretation:

- **Digits such as 0, 1, and 6 achieved almost perfect classification.**
- **Slightly lower precision for digits such as 5 and 8 indicates occasional confusion between visually similar digits.**
- Overall performance across all classes remains highly balanced.

Macro Average:

- **Precision: 0.99**
- **Recall: 0.99**

- **F1-score: 0.99**

Weighted Average:

- **Precision: 0.99**
- **Recall: 0.99**
- **F1-score: 0.99**

Interpretation:

The CNN model effectively learned spatial features such as edges, curves, and shapes from handwritten digits. Convolution layers captured local patterns while pooling layers reduced dimensionality, enabling efficient classification.

High accuracy indicates that CNN architectures are well suited for image recognition tasks.

7. Hyperparameter Configuration

Model parameters used:

- Convolution Layer 1: 32 filters, kernel size 3×3
- Convolution Layer 2: 64 filters, kernel size 3×3
- Pooling Layer: MaxPooling 2×2
- Fully Connected Layer: 128 neurons
- Dropout Rate: 0.3
- Output Layer: 10 neurons with softmax activation

Training parameters:

- Optimizer: Adam
- Loss Function: Sparse Categorical Cross Entropy

- **Epochs: 5**
- Batch Size: 64

These hyperparameters allowed the model to achieve high classification accuracy while preventing overfitting.

OUTPUT

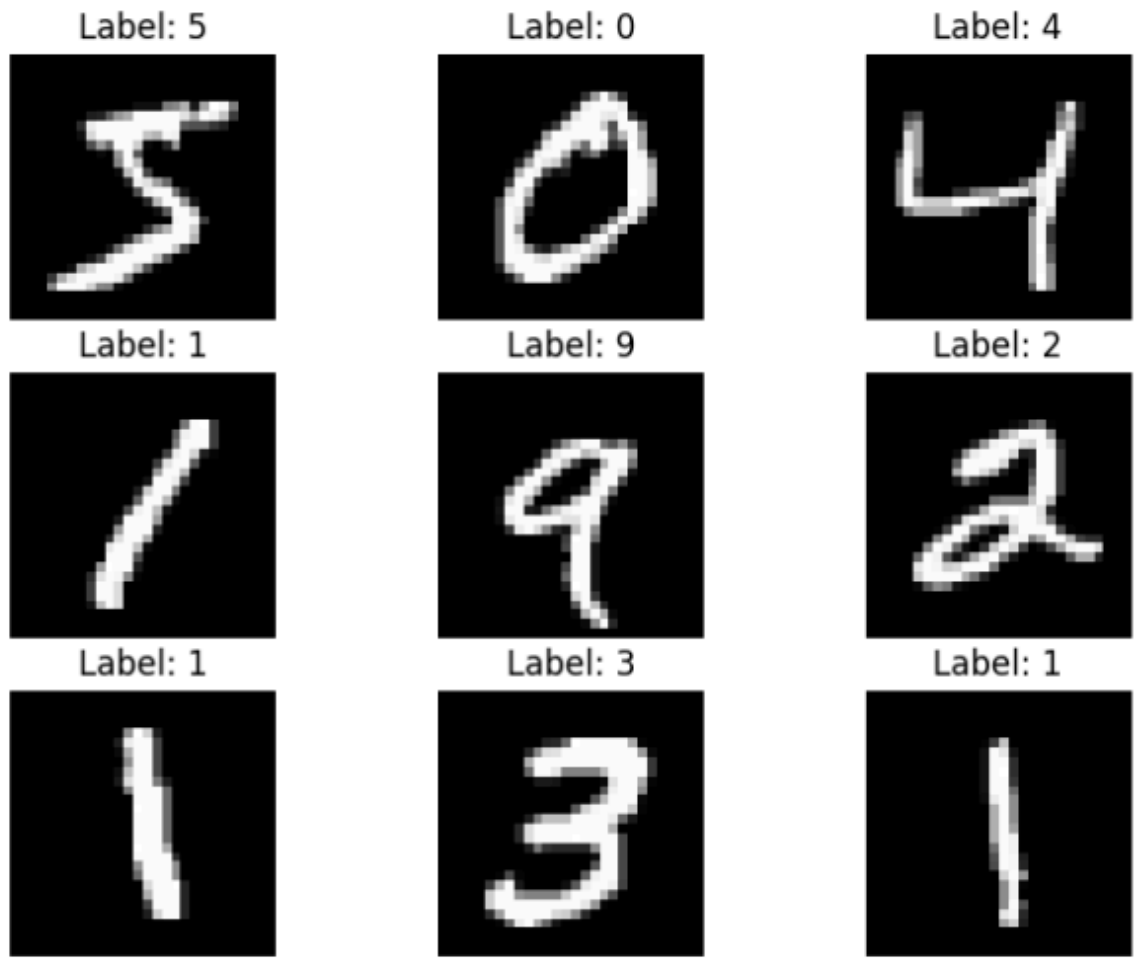
- CNN successfully classified handwritten digits from the MNIST dataset.
- The model achieved an overall accuracy of approximately 99 percent.
- Classification metrics show consistently high precision, recall, and F1-score across all digit classes.
- Visualization of predictions confirmed the model's ability to correctly recognize handwritten digits.

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from tensorflow import keras
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout

from sklearn.metrics import confusion_matrix, classification_report
```

```
from tensorflow.keras.datasets import mnist

(X_train, y_train), (X_test, y_test) = mnist.load_data()

print("Training data shape:", X_train.shape)
print("Testing data shape:", X_test.shape)
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Training data shape: (60000, 28, 28)
Testing data shape: (10000, 28, 28)
```

```python
plt.figure(figsize=(8,6))

for i in range(9):
    plt.subplot(3,3,i+1)
    plt.imshow(X_train[i], cmap='gray')
    plt.title("Label: " + str(y_train[i]))
    plt.axis('off')

plt.show()
```



```python
X_train = X_train / 255.0
X_test = X_test / 255.0
```

```python
X_train = X_train.reshape(-1,28,28,1)
X_test = X_test.reshape(-1,28,28,1)

print("New training shape:", X_train.shape)
```

```
New training shape: (60000, 28, 28, 1)
```

```python
model = Sequential()

model.add(Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)))
model.add(MaxPooling2D((2,2)))

model.add(Conv2D(64, (3,3), activation='relu'))
model.add(MaxPooling2D((2,2)))

model.add(Flatten())

model.add(Dense(128, activation='relu'))
model.add(Dropout(0.3))

model.add(Dense(10, activation='softmax'))

model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

model.summary()
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential"
```

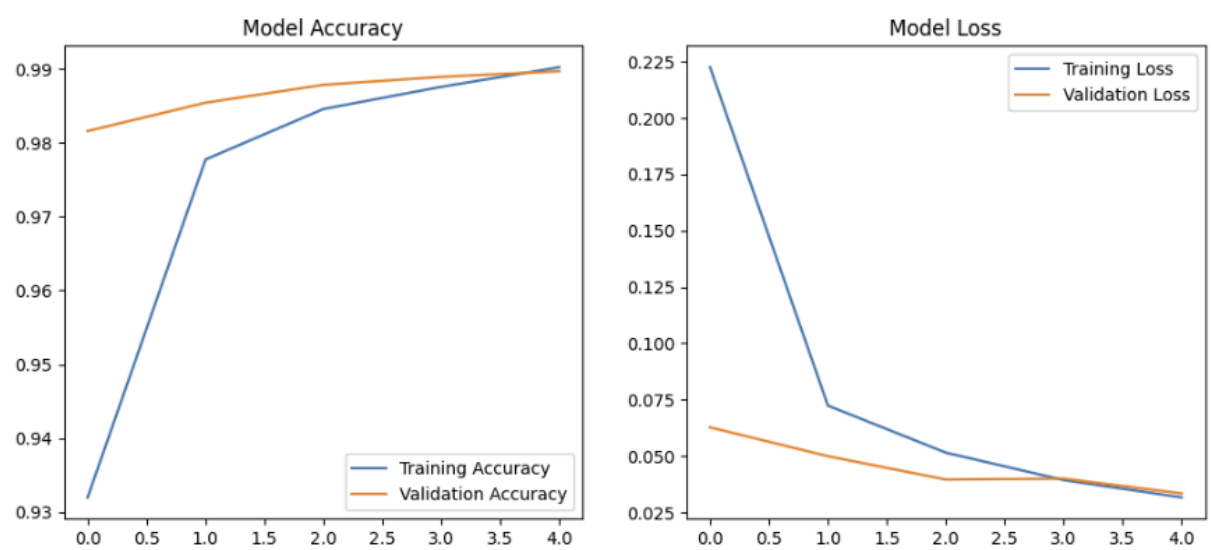
Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_1 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten (Flatten)	(None, 1600)	0
dense (Dense)	(None, 128)	204,928
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 10)	1,290

```
Total params: 225,034 (879.04 KB)
Trainable params: 225,034 (879.04 KB)
Non-trainable params: 0 (0.00 B)
```

```
history = model.fit(
    X_train,
    y_train,
    epochs=5,
    batch_size=64,
    validation_split=0.2
)
```

```
Epoch 1/5
750/750 ━━━━━━━━━━━━━━━━━━━━ 42s 54ms/step - accuracy: 0.9320 - loss: 0.2224 - val_accuracy: 0.9816 - val_loss: 0.0628
Epoch 2/5
750/750 ━━━━━━━━━━━━━━━━━━━━ 42s 56ms/step - accuracy: 0.9777 - loss: 0.0724 - val_accuracy: 0.9854 - val_loss: 0.0500
Epoch 3/5
750/750 ━━━━━━━━━━━━━━━━━━━━ 80s 54ms/step - accuracy: 0.9846 - loss: 0.0516 - val_accuracy: 0.9878 - val_loss: 0.0396
Epoch 4/5
750/750 ━━━━━━━━━━━━━━━━━━━━ 41s 54ms/step - accuracy: 0.9876 - loss: 0.0394 - val_accuracy: 0.9889 - val_loss: 0.0401
Epoch 5/5
750/750 ━━━━━━━━━━━━━━━━━━━━ 40s 54ms/step - accuracy: 0.9902 - loss: 0.0317 - val_accuracy: 0.9897 - val_loss: 0.0335
```

```
plt.figure(figsize=(12,5))

plt.subplot(1,2,1)
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.legend()
plt.title("Model Accuracy")

plt.subplot(1,2,2)
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.legend()
plt.title("Model Loss")

plt.show()
```



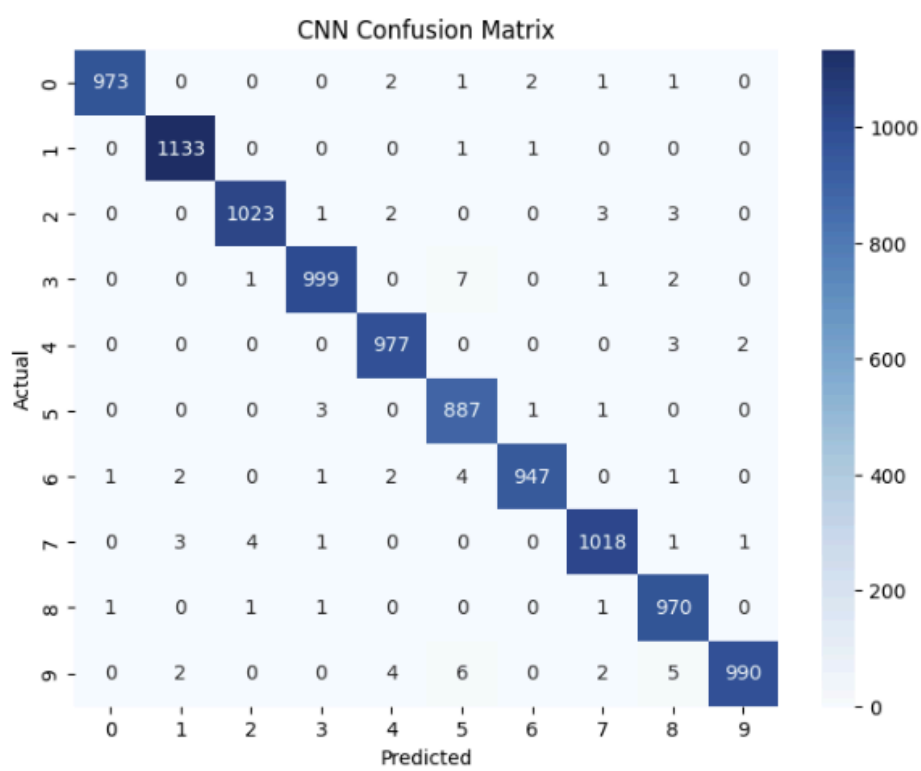
```
test_loss, test_acc = model.evaluate(X_test, y_test)

print("Test Accuracy:", test_acc)
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 3s 9ms/step - accuracy: 0.9917 - loss: 0.0256
Test Accuracy: 0.9916999936103821
```

```
y_pred_prob = model.predict(X_test)
y_pred = np.argmax(y_pred_prob, axis=1)
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 3s 9ms/step
```

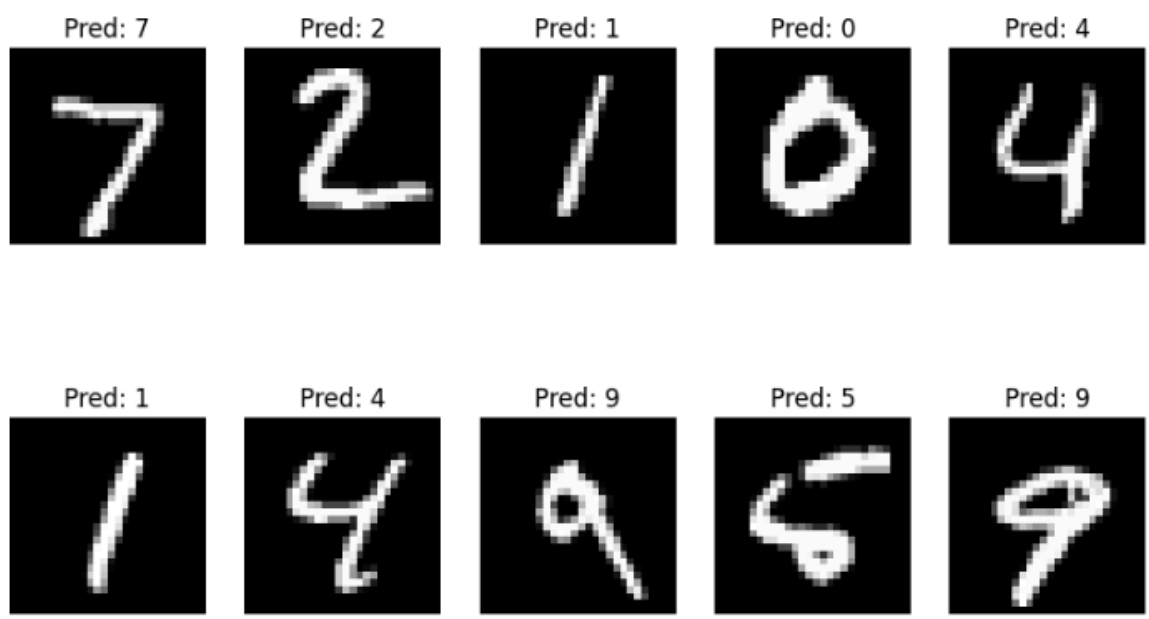
```
cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(8,6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("CNN Confusion Matrix")

plt.show()
```



```python
print(classification_report(y_test, y_pred))
```

```
              precision    recall  f1-score   support

           0       1.00      0.99      1.00       980
           1       0.99      1.00      1.00      1135
           2       0.99      0.99      0.99      1032
           3       0.99      0.99      0.99      1010
           4       0.99      0.99      0.99       982
           5       0.98      0.99      0.99       892
           6       1.00      0.99      0.99       958
           7       0.99      0.99      0.99      1028
           8       0.98      1.00      0.99       974
           9       1.00      0.98      0.99      1009

    accuracy                           0.99     10000
   macro avg       0.99      0.99      0.99     10000
weighted avg       0.99      0.99      0.99     10000
```

```python
plt.figure(figsize=(10,6))

for i in range(10):
    plt.subplot(2,5,i+1)
    plt.imshow(X_test[i].reshape(28,28), cmap='gray')
    plt.title("Pred: " + str(y_pred[i]))
    plt.axis('off')

plt.show()
```



CONCLUSION

- Convolutional Neural Networks are highly effective for image classification tasks.
- The model successfully learned spatial patterns from handwritten digit images.
- High classification accuracy demonstrates the capability of CNN architectures to capture complex visual features.
- Convolution and pooling layers enable automatic feature extraction without manual feature engineering.
- The experiment confirms the effectiveness of deep learning techniques for computer vision applications